Technical Assessment – Python Data Scraper Internship

Position
Software Engineering Intern – Python & Data Processing

Duration
5 days (recommended)

Difficulty
Junior / Intern

Objective
Develop a web application that allows a user to upload documents (PDF and other supported formats), automatically extracts textual information, processes it, and stores the extracted data in a database.

Evaluate: clean Python code, application architecture, databases, document parsing, data preprocessing, technical decisions, and responsible AI usage.

AI tools (ChatGPT, Claude, Gemini, Cursor, Copilot, etc.) are allowed, but you must explain which tools/functions were used, why, and how they work.

Project Requirements
Backend: Python (Flask/FastAPI recommended, Django optional).
Frontend: Upload page (upload button, supported types, status, extract button) and Results page (extracted text, metadata, DB ID, processing time).
Supported Files: PDF, DOCX, TXT. Bonus: Images (OCR), CSV, Excel.

Document Processing
Read → Extract Text → Clean Text.
Examples: remove extra spaces, duplicated blank lines, normalize Unicode, lowercase (optional), remove unnecessary symbols.

Database
Store: Document ID, Original Filename, Upload Date, File Type, Extracted Text, Word Count, Character Count.
Choose SQLite, PostgreSQL or MySQL and justify your choice during the presentation.

Application Workflow
Upload File → Validate File → Extract Text → Preprocess Text → Store in Database → Display Result.

Suggested Folder Structure
```
project/
■■■ app.py
■■■ database.py
■■■ scraper/
■   ■■■ pdf_reader.py
■   ■■■ docx_reader.py
■   ■■■ txt_reader.py
■■■ templates/
■■■ static/
■■■ uploads/
■■■ models/
```

■■■ `requirements.txt`
■■■ `README.md`

Technical Expectations
Functions, modular design, optional classes, error handling, meaningful variable names, comments where needed.

Data Preprocessing
Remove multiple spaces, empty lines, tabs, normalize line endings, count words and characters.

AI Usage
Explain prompts used, generated code, modified code, and why AI suggestions were accepted or rejected.

Bonus Features
OCR (Tesseract, EasyOCR, PaddleOCR), Search, AI Summarization, Keyword Extraction, Duplicate Detection, REST API (POST /upload, GET /documents, GET /documents/{id}, DELETE /documents/{id}), Export JSON/CSV.

Expected Deliverables
Source code, README, requirements.txt, sample database, and at least three sample documents.

README Must Include
Installation, execution instructions, technologies used, AI tools used, architecture, database justification, future improvements.

Evaluation Rubric (100 Points)
Project Structure (10), Python Code Quality (15), Document Parsing (15), Data Preprocessing (10), Database Design (10), Web Interface (10), Error Handling (10), Documentation (5), AI Usage (10), Presentation (5).

Technical Interview Questions
Data Preprocessing, Optimization & Regularization, Data Structures, Database Systems, Static vs Dynamic Libraries, Attention Mechanisms, Choosing the Right Database Type.

Evaluation Criteria
Demonstrate maintainable software design, clean modular Python, correct preprocessing, informed technical decisions, effective AI usage, and the ability to explain implementation choices.